In [1]:
```
from importlib.metadata import version  # 从 importlib.metadata 模块导入 version

pkgs = [
    "huggingface_hub",  # huggingface_hub 包，通常用于下载预训练权重和模型文件
    "tokenizers",       # tokenizers 包，通常用于实现高效的文本分词器
    "torch",            # torch 包（PyTorch），通常用于实现和运行深度学习模型
]
for p in pkgs:  # 遍历 pkgs 列表中的每个包名
    # 打印包名及其对应的版本号。version(p) 函数会查找并返回指定包名的版本。
    print(f"{p} version: {version(p)}")
```

```
huggingface_hub version: 0.36.2
tokenizers version: 0.22.2
torch version: 2.6.0+cpu
```

In [2]:
```
USE_BASE_MODEL = False # 预测下一个词（Next Token Prediction）
USE_REASONING_MODEL = True # 提升逻辑推理能力和复杂问题解决能力
USE_INSTRUCT_MODEL = False # 遵循用户指令、生成格式化的回答

if (USE_BASE_MODEL + USE_REASONING_MODEL
    + USE_INSTRUCT_MODEL) != 1:
    raise AttributeError("Only one of the options above can be True.")
```

In [3]:
```
import torch
import torch.nn as nn


class FeedForward(nn.Module):
    # 这个模块实现了一个带有 SiLU 激活和 SwiGLU 结构的 前馈网络（FFN），这是现代
    def __init__(self, cfg):
        super().__init__()
        self.fc1 = nn.Linear(cfg["emb_dim"], cfg["hidden_dim"], dtype=cfg["dtype
        self.fc2 = nn.Linear(cfg["emb_dim"], cfg["hidden_dim"], dtype=cfg["dtype
        self.fc3 = nn.Linear(cfg["hidden_dim"], cfg["emb_dim"], dtype=cfg["dtype

    def forward(self, x):
        x_fc1 = self.fc1(x)
        x_fc2 = self.fc2(x)
        x = nn.functional.silu(x_fc1) * x_fc2
        return self.fc3(x)
```

In [4]:
```
class RMSNorm(nn.Module):
    # RMSNorm (Root Mean Square Normalization) 是一种用于 Transformer 模型的高效
    def __init__(self, emb_dim, eps=1e-6, bias=False, qwen3_compatible=True):
        super().__init__()
        self.eps = eps
        self.qwen3_compatible = qwen3_compatible
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim)) if bias else None

    def forward(self, x):
        input_dtype = x.dtype

        if self.qwen3_compatible:
            x = x.to(torch.float32)

        # 1. 计算均方 (Mean Square)
        # x.pow(2): 对输入张量的所有元素进行平方
```

```
        # .mean(dim=-1, keepdim=True): 沿着最后一个维度（特征维度/emb_dim）求平均
        # variance 的形状是 (batch_size, seq_len, 1)，用于广播。
        variance = x.pow(2).mean(dim=-1, keepdim=True)

        # 2. 归一化 (Normalization)
        # torch.rsqrt(variance + self.eps): 计算均方的平方根的倒数 (Reciprocal S
        # RMS = sqrt(variance) -> 归一化因子 = 1 / RMS
        # norm_x = x / RMS，即 x * (1 / RMS)
        norm_x = x * torch.rsqrt(variance + self.eps)

        # 3. 缩放 (Scaling)
        # 将归一化后的结果乘以可学习的缩放参数 self.scale（在 emb_dim 维度上广播）
        norm_x = norm_x * self.scale

        # 4. 偏置 (Shifting, 如果启用)
        if self.shift is not None:
            # 如果存在偏置参数，则将其加到归一化后的结果上
            norm_x = norm_x + self.shift

        return norm_x.to(input_dtype)
```

In [5]:

```
# 旋转位置嵌入 (Rotary Positional Embedding, RoPE)
def compute_rope_params(head_dim, theta_base=10_000, context_length=4096, dtype=
    # ---------------------------------------------------------------------
    # head_dim: 单个注意力头（Attention Head）的特征维度。
    # theta_base: 用于计算频率的基数，通常取 10000，源自原始 RoPE 论文。
    # context_length: 模型支持的最大序列长度。
    # dtype: 计算张量的数据类型。
    # ---------------------------------------------------------------------

    assert head_dim % 2 == 0, "Embedding dimension must be even"

    # Compute the inverse frequencies
    inv_freq = 1.0 / (theta_base ** (torch.arange(0, head_dim, 2, dtype=dtype)[:

    # Generate position indices
    positions = torch.arange(context_length, dtype=dtype)

    # Compute the angles
    angles = positions.unsqueeze(1) * inv_freq.unsqueeze(0)  # Shape: (context_l

    # Expand angles to match the head_dim
    angles = torch.cat([angles, angles], dim=1)  # Shape: (context_length, head_

    # Precompute sine and cosine
    cos = torch.cos(angles)
    sin = torch.sin(angles)

    return cos, sin


def apply_rope(x, cos, sin):
    # ---------------------------------------------------------------------
    # x: 待旋转的张量（通常是 Query 或 Key 向量），形状: (batch_size, num_heads,
    # cos: 余弦矩阵，形状: (context_length, head_dim)
    # sin: 正弦矩阵，形状: (context_length, head_dim)
    # ---------------------------------------------------------------------

    # x: (batch_size, num_heads, seq_len, head_dim)
    batch_size, num_heads, seq_len, head_dim = x.shape
```

```
    assert head_dim % 2 == 0, "Head dimension must be even"

    # Split x into first half and second half
    x1 = x[..., : head_dim // 2]  # First half
    x2 = x[..., head_dim // 2 :]  # Second half

    # Adjust sin and cos shapes
    cos = cos[:seq_len, :].unsqueeze(0).unsqueeze(0)  # Shape: (1, 1, seq_len, h
    sin = sin[:seq_len, :].unsqueeze(0).unsqueeze(0)

    # Apply the rotary transformation
    rotated = torch.cat((-x2, x1), dim=-1)
    x_rotated = (x * cos) + (rotated * sin)

    # It's ok to use lower-precision after applying cos and sin rotation
    return x_rotated.to(dtype=x.dtype)
```

In [6]:
```
# GQA 是一种高效的注意力机制，通过重用 Key (K) 和 Value (V) 投影，显著减少了内存
class GroupedQueryAttention(nn.Module):
    def __init__(
        self, d_in, num_heads, num_kv_groups, head_dim=None, qk_norm=False, dtyp
    ):
        # d_in: 输入的嵌入维度（模型维度，如 768, 1024 等）
        # num_heads: Query (Q) 的注意力头总数
        # num_kv_groups: Key (K) 和 Value (V) 的组数，也即 K 和 V 的实际头数
        # head_dim: 每个注意力头的维度（如果未指定，则从 d_in / num_heads 计算）
        # qk_norm: 布尔值，是否在 Q 和 K 上应用 RMSNorm (Post-Attention Normaliz
        # dtype: 张量的数据类型 (如 torch.bfloat16)
        super().__init__()
        assert num_heads % num_kv_groups == 0, "num_heads must be divisible by n

        self.num_heads = num_heads
        self.num_kv_groups = num_kv_groups
        self.group_size = num_heads // num_kv_groups

        if head_dim is None:
            assert d_in % num_heads == 0, "`d_in` must be divisible by `num_head
            head_dim = d_in // num_heads

        self.head_dim = head_dim
        self.d_out = num_heads * head_dim

        self.W_query = nn.Linear(d_in, self.d_out, bias=False, dtype=dtype)
        self.W_key = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype
        self.W_value = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dty

        self.out_proj = nn.Linear(self.d_out, d_in, bias=False, dtype=dtype)

        if qk_norm:
            self.q_norm = RMSNorm(head_dim, eps=1e-6)
            self.k_norm = RMSNorm(head_dim, eps=1e-6)
        else:
            self.q_norm = self.k_norm = None

    def forward(self, x, mask, cos, sin):
        b, num_tokens, _ = x.shape

        # Apply projections
        queries = self.W_query(x)  # (b, num_tokens, num_heads * head_dim)
        keys = self.W_key(x)       # (b, num_tokens, num_kv_groups * head_dim)
```

```
        values = self.W_value(x)   # (b, num_tokens, num_kv_groups * head_dim)

        # Reshape
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim).tra
        keys = keys.view(b, num_tokens, self.num_kv_groups, self.head_dim).trans
        values = values.view(b, num_tokens, self.num_kv_groups, self.head_dim).t

        # Optional normalization
        if self.q_norm:
            queries = self.q_norm(queries)
        if self.k_norm:
            keys = self.k_norm(keys)

        # Apply RoPE
        queries = apply_rope(queries, cos, sin)
        keys = apply_rope(keys, cos, sin)

        # Expand K and V to match number of heads
        keys = keys.repeat_interleave(self.group_size, dim=1)
        values = values.repeat_interleave(self.group_size, dim=1)

        # Attention
        attn_scores = queries @ keys.transpose(2, 3)
        attn_scores = attn_scores.masked_fill(mask, -torch.inf)
        attn_weights = torch.softmax(attn_scores / self.head_dim**0.5, dim=-1)

        context = (attn_weights @ values).transpose(1, 2).reshape(b, num_tokens,
        return self.out_proj(context)
```

In [7]:

```
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = GroupedQueryAttention(
            d_in=cfg["emb_dim"],
            num_heads=cfg["n_heads"],
            head_dim=cfg["head_dim"],
            num_kv_groups=cfg["n_kv_groups"],
            qk_norm=cfg["qk_norm"],
            dtype=cfg["dtype"]
        )
        self.ff = FeedForward(cfg)
        self.norm1 = RMSNorm(cfg["emb_dim"], eps=1e-6)
        self.norm2 = RMSNorm(cfg["emb_dim"], eps=1e-6)

    def forward(self, x, mask, cos, sin):
        # Shortcut connection for attention block
        shortcut = x
        x = self.norm1(x)
        x = self.att(x, mask, cos, sin)  # Shape [batch_size, num_tokens, emb_si
        x = x + shortcut  # Add the original input back

        # Shortcut connection for feed-forward block
        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = x + shortcut  # Add the original input back

        return x
```

In [8]:
```
class Qwen3Model(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        # Main model parameters
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"], dtype=cfg

        self.trf_blocks = nn.ModuleList(  # ModuleList since Sequential can only
            [TransformerBlock(cfg) for _ in range(cfg["n_layers"])]
        )

        self.final_norm = RMSNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False,

        # Reusuable utilities
        if cfg["head_dim"] is None:
            head_dim = cfg["emb_dim"] // cfg["n_heads"]
        else:
            head_dim = cfg["head_dim"]
        cos, sin = compute_rope_params(
            head_dim=head_dim,
            theta_base=cfg["rope_base"],
            context_length=cfg["context_length"]
        )
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)
        self.cfg = cfg


    def forward(self, in_idx):
        # Forward pass
        tok_embeds = self.tok_emb(in_idx)
        x = tok_embeds

        num_tokens = x.shape[1]
        mask = torch.triu(torch.ones(num_tokens, num_tokens, device=x.device, dt

        for block in self.trf_blocks:
            x = block(x, mask, self.cos, self.sin)
        x = self.final_norm(x)
        logits = self.out_head(x.to(self.cfg["dtype"]))
        return logits
```

2. Initialize model

In [9]:
```
CHOOSE_MODEL = "0.6B"

if CHOOSE_MODEL == "0.6B":
    QWEN3_CONFIG = {
        "vocab_size": 151_936,           # Vocabulary size
        "context_length": 40_960,        # Context length that was used to train
        "emb_dim": 1024,                 # Embedding dimension
        "n_heads": 16,                   # Number of attention heads
        "n_layers": 28,                  # Number of layers
        "hidden_dim": 3072,              # Size of the intermediate dimension in
```

```
        "head_dim": 128,                # Size of the heads in GQA
        "qk_norm": True,                # Whether to normalize queries and keys
        "n_kv_groups": 8,               # Key-Value groups for grouped-query at
        "rope_base": 1_000_000.0,       # The base in RoPE's "theta"
        "dtype": torch.bfloat16,        # Lower-precision dtype to reduce memor
    }

elif CHOOSE_MODEL == "1.7B":
    QWEN3_CONFIG = {
        "vocab_size": 151_936,
        "context_length": 40_960,
        "emb_dim": 2048,                # 2x larger than above
        "n_heads": 16,
        "n_layers": 28,
        "hidden_dim": 6144,             # 2x larger than above
        "head_dim": 128,
        "qk_norm": True,
        "n_kv_groups": 8,
        "rope_base": 1_000_000.0,
        "dtype": torch.bfloat16,
    }

elif CHOOSE_MODEL == "4B":
    QWEN3_CONFIG = {
        "vocab_size": 151_936,
        "context_length": 40_960,
        "emb_dim": 2560,                # 25% larger than above
        "n_heads": 32,                  # 2x larger than above
        "n_layers": 36,                 # 29% larger than above
        "hidden_dim": 9728,             # ~3x larger than above
        "head_dim": 128,
        "qk_norm": True,
        "n_kv_groups": 8,
        "rope_base": 1_000_000.0,
        "dtype": torch.bfloat16,
    }

elif CHOOSE_MODEL == "8B":
    QWEN3_CONFIG = {
        "vocab_size": 151_936,
        "context_length": 40_960,
        "emb_dim": 4096,                # 60% larger than above
        "n_heads": 32,
        "n_layers": 36,                 # 26% larger than above
        "hidden_dim": 12288,
        "head_dim": 128,
        "qk_norm": True,
        "n_kv_groups": 8,
        "rope_base": 1_000_000.0,
        "dtype": torch.bfloat16,
    }

elif CHOOSE_MODEL == "14B":
    QWEN3_CONFIG = {
        "vocab_size": 151_936,
        "context_length": 40_960,
        "emb_dim": 5120,                # 25% larger than above
        "n_heads": 40,                  # 25% larger than above
        "n_layers": 40,                 # 11% larger than above
        "hidden_dim": 17408,            # 42% larger than above
```

```
        "head_dim": 128,
        "qk_norm": True,
        "n_kv_groups": 8,
        "rope_base": 1_000_000.0,
        "dtype": torch.bfloat16,
    }

elif CHOOSE_MODEL == "32B":
    QWEN3_CONFIG = {
        "vocab_size": 151_936,
        "context_length": 40_960,
        "emb_dim": 5120,
        "n_heads": 64,                    # 60% larger than above
        "n_layers": 64,                   # 60% larger than above
        "hidden_dim": 25600,              # 47% larger than above
        "head_dim": 128,
        "qk_norm": True,
        "n_kv_groups": 8,
        "rope_base": 1_000_000.0,
        "dtype": torch.bfloat16,
    }

else:
    raise ValueError(f"{CHOOSE_MODEL} is not supported.")
```

In [10]:
```
torch.manual_seed(123)
model = Qwen3Model(QWEN3_CONFIG)
```

In [11]:
```
model
```

Out[11]:
```
Qwen3Model(
  (tok_emb): Embedding(151936, 1024)
  (trf_blocks): ModuleList(
    (0-27): 28 x TransformerBlock(
      (att): GroupedQueryAttention(
        (W_query): Linear(in_features=1024, out_features=2048, bias=False)
        (W_key): Linear(in_features=1024, out_features=1024, bias=False)
        (W_value): Linear(in_features=1024, out_features=1024, bias=False)
        (out_proj): Linear(in_features=2048, out_features=1024, bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): FeedForward(
        (fc1): Linear(in_features=1024, out_features=3072, bias=False)
        (fc2): Linear(in_features=1024, out_features=3072, bias=False)
        (fc3): Linear(in_features=3072, out_features=1024, bias=False)
      )
      (norm1): RMSNorm()
      (norm2): RMSNorm()
    )
  )
  (final_norm): RMSNorm()
  (out_head): Linear(in_features=1024, out_features=151936, bias=False)
)
```

- A quick check that the forward pass works before continuing:

In [12]:
```
model(torch.tensor([1, 2, 3]).unsqueeze(0))
```

Out[12]:
```
tensor([[[-0.2305, -0.0153, -0.6992,  ...,  0.4453,  0.1221,  1.0781],
         [-0.6523,  0.5352, -0.0757,  ..., -0.0620,  0.5391,  0.3125],
         [-0.4824, -0.1572,  0.1084,  ..., -0.2227,  0.2383,  0.6289]]],
       dtype=torch.bfloat16, grad_fn=<UnsafeViewBackward0>)
```

In [13]:
```python
total_params = sum(p.numel() for p in model.parameters())
print(f"Total number of parameters: {total_params:,}")

# Account for weight tying
total_params_normalized = total_params - model.tok_emb.weight.numel()
print(f"\nTotal number of unique parameters: {total_params_normalized:,}")
```

```
Total number of parameters: 751,632,384

Total number of unique parameters: 596,049,920
```

In [14]:
```python
def model_memory_size(model, input_dtype=torch.float32):
    total_params = 0
    total_grads = 0
    for param in model.parameters():
        # Calculate total number of elements per parameter
        param_size = param.numel()
        total_params += param_size
        # Check if gradients are stored for this parameter
        if param.requires_grad:
            total_grads += param_size

    # Calculate buffer size (non-parameters that require memory)
    total_buffers = sum(buf.numel() for buf in model.buffers())

    # Size in bytes = (Number of elements) * (Size of each element in bytes)
    # We assume parameters and gradients are stored in the same type as input dt
    element_size = torch.tensor(0, dtype=input_dtype).element_size()
    total_memory_bytes = (total_params + total_grads + total_buffers) * element_

    # Convert bytes to gigabytes
    total_memory_gb = total_memory_bytes / (1024**3)

    return total_memory_gb

print(f"float32 (PyTorch default): {model_memory_size(model, input_dtype=torch.f
print(f"bfloat16: {model_memory_size(model, input_dtype=torch.bfloat16):.2f} GB"
```

```
float32 (PyTorch default): 5.64 GB
bfloat16: 2.82 GB
```

In [15]:
```python
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

model.to(device);
```

In [16]:
```python
def load_weights_into_qwen(model, param_config, params):
    def assign(left, right, tensor_name="unknown"):
        if left.shape != right.shape:
            raise ValueError(f"Shape mismatch in tensor '{tensor_name}'. Left: {
```

```
        with torch.no_grad():
            if isinstance(right, torch.Tensor):
                left.copy_(right)
            else:
                left.copy_(torch.as_tensor(right, dtype=left.dtype, device=left.

        return left

    model.tok_emb.weight = assign(model.tok_emb.weight, params["model.embed_toke

    for l in range(param_config["n_layers"]):
        block = model.trf_blocks[l]
        att = block.att

        # Q, K, V projections
        att.W_query.weight = assign(
            att.W_query.weight,
            params[f"model.layers.{l}.self_attn.q_proj.weight"],
            f"model.layers.{l}.self_attn.q_proj.weight"
        )
        att.W_key.weight = assign(
            att.W_key.weight,
            params[f"model.layers.{l}.self_attn.k_proj.weight"],
            f"model.layers.{l}.self_attn.k_proj.weight"
        )
        att.W_value.weight = assign(
            att.W_value.weight,
            params[f"model.layers.{l}.self_attn.v_proj.weight"],
            f"model.layers.{l}.self_attn.v_proj.weight"
        )

        # Output projection
        att.out_proj.weight = assign(
            att.out_proj.weight,
            params[f"model.layers.{l}.self_attn.o_proj.weight"],
            f"model.layers.{l}.self_attn.o_proj.weight"
        )

        # QK norms
        if hasattr(att, "q_norm") and att.q_norm is not None:
            att.q_norm.scale = assign(
                att.q_norm.scale,
                params[f"model.layers.{l}.self_attn.q_norm.weight"],
                f"model.layers.{l}.self_attn.q_norm.weight"
            )
        if hasattr(att, "k_norm") and att.k_norm is not None:
            att.k_norm.scale = assign(
                att.k_norm.scale,
                params[f"model.layers.{l}.self_attn.k_norm.weight"],
                f"model.layers.{l}.self_attn.k_norm.weight"
            )

        # Attention layernorm
        block.norm1.scale = assign(
            block.norm1.scale,
            params[f"model.layers.{l}.input_layernorm.weight"],
            f"model.layers.{l}.input_layernorm.weight"
        )

        # Feedforward weights
```

```
        block.ff.fc1.weight = assign(
            block.ff.fc1.weight,
            params[f"model.layers.{l}.mlp.gate_proj.weight"],
            f"model.layers.{l}.mlp.gate_proj.weight"
        )
        block.ff.fc2.weight = assign(
            block.ff.fc2.weight,
            params[f"model.layers.{l}.mlp.up_proj.weight"],
            f"model.layers.{l}.mlp.up_proj.weight"
        )
        block.ff.fc3.weight = assign(
            block.ff.fc3.weight,
            params[f"model.layers.{l}.mlp.down_proj.weight"],
            f"model.layers.{l}.mlp.down_proj.weight"
        )
        block.norm2.scale = assign(
            block.norm2.scale,
            params[f"model.layers.{l}.post_attention_layernorm.weight"],
            f"model.layers.{l}.post_attention_layernorm.weight"
        )

    # Final normalization and output head
    model.final_norm.scale = assign(model.final_norm.scale, params["model.norm.w

    if "lm_head.weight" in params:
        model.out_head.weight = assign(model.out_head.weight, params["lm_head.we
    else:
        model.out_head.weight = model.tok_emb.weight
        print("Model uses weight tying.")
```

In [17]:

```
import json
import os
from pathlib import Path
from safetensors.torch import load_file


# 本地已经下载好的 Qwen3-0.6B 模型目录
local_dir = Path(r"D:\桌面\typora文件\八斗AI\models\Qwen\Qwen3-0.6B")

# 保留 repo_id，后面的 tokenizer 代码会用到它来判断是否是 Base 模型
if USE_REASONING_MODEL or USE_INSTRUCT_MODEL:
    repo_id = f"Qwen/Qwen3-{CHOOSE_MODEL}"
else:
    repo_id = f"Qwen/Qwen3-{CHOOSE_MODEL}-Base"


# Qwen3-0.6B 通常是单个 model.safetensors 文件
weights_file = local_dir / "model.safetensors"

if not weights_file.exists():
    raise FileNotFoundError(f"找不到模型权重文件：{weights_file}")

weights_dict = load_file(str(weights_file))

load_weights_into_qwen(model, QWEN3_CONFIG, weights_dict)
model.to(device)
del weights_dict
```

4. Load tokenizer

In [18]:

```
import re
from tokenizers import Tokenizer

class Qwen3Tokenizer:
    _SPECIALS = [
        "<|endoftext|>",
        "<|im_start|>", "<|im_end|>",
        "<|object_ref_start|>", "<|object_ref_end|>",
        "<|box_start|>", "<|box_end|>",
        "<|quad_start|>", "<|quad_end|>",
        "<|vision_start|>", "<|vision_end|>",
        "<|vision_pad|>", "<|image_pad|>", "<|video_pad|>",
        "<think>", "</think>"
    ]
    _SPLIT_RE = re.compile(r"(<\|[^>]+?\|>|<think>|</think>)")

    def __init__(self, tokenizer_file_path="tokenizer.json", repo_id=None,
                 apply_chat_template=True, add_generation_prompt=False, add_thin

        self.apply_chat_template = apply_chat_template
        self.add_generation_prompt = add_generation_prompt
        self.add_thinking = add_thinking

        tok_file = Path(tokenizer_file_path)
        self._tok = Tokenizer.from_file(str(tok_file))
        self._special_to_id = {}
        for t in self._SPECIALS:
            tid = self._tok.token_to_id(t)
            if tid is not None:
                self._special_to_id[t] = tid

        self.pad_token_id = self._special_to_id["<|endoftext|>"]
        self.eos_token_id = self.pad_token_id

        if repo_id and "Base" not in repo_id:
            eos_token = "<|im_end|>"
        else:
            eos_token = "<|endoftext|>"
        if eos_token in self._special_to_id:
            self.eos_token_id = self._special_to_id[eos_token]

    def encode(self, text, chat_wrapped=None):
        if chat_wrapped is None:
            chat_wrapped = self.apply_chat_template

        stripped = text.strip()
        if stripped in self._special_to_id and "\n" not in stripped:
            return [self._special_to_id[stripped]]

        if chat_wrapped:
            text = self._wrap_chat(text)

        ids = []
        for part in filter(None, self._SPLIT_RE.split(text)):
            if part in self._special_to_id:
```

```
                ids.append(self._special_to_id[part])
            else:
                ids.extend(self._tok.encode(part).ids)
        return ids

    def decode(self, ids):
        return self._tok.decode(ids, skip_special_tokens=False)

    def _wrap_chat(self, user_msg):
        s = f"<|im_start|>user\n{user_msg}<|im_end|>\n"
        if self.add_generation_prompt:
            s += "<|im_start|>assistant"
            if self.add_thinking:
                s += "\n"
            else:
                s += "\n<think>\n\n</think>\n\n"
        return s
```

In [19]:
```
# 直接使用本地 tokenizer.json
tokenizer_file_path = local_dir / "tokenizer.json"

if not tokenizer_file_path.exists():
    raise FileNotFoundError(f"找不到 tokenizer 文件：{tokenizer_file_path}")


if USE_REASONING_MODEL or USE_INSTRUCT_MODEL:
    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=str(tokenizer_file_path),
        repo_id=repo_id,
        apply_chat_template=True,
        add_generation_prompt=True,
        add_thinking=USE_REASONING_MODEL
    )

else:
    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=str(tokenizer_file_path),
        repo_id=repo_id,
        apply_chat_template=False,
        add_generation_prompt=False,
        add_thinking=False
    )
```

In [20]:
```
prompt = "Give me a short introduction to MTP(Multi Token Prediction)"

input_token_ids = tokenizer.encode(prompt)
text = tokenizer.decode(input_token_ids)
text
```

Out[20]:
```
'<|im_start|>user\nGive me a short introduction to MTP(Multi Token Prediction)<
|im_end|>\n<|im_start|>assistant\n'
```

In [21]:
```
def generate_text_basic_stream(model, token_ids, max_new_tokens, eos_token_id=No

    model.eval()
    with torch.no_grad():
        for _ in range(max_new_tokens):
            out = model(token_ids)[:, -1]
            next_token = torch.argmax(out, dim=-1, keepdim=True)
```

```
            if (eos_token_id is not None
                    and torch.all(next_token == eos_token_id)):
                break

            yield next_token

            token_ids = torch.cat([token_ids, next_token], dim=1)
```

In [22]:
```
input_token_ids_tensor = torch.tensor(input_token_ids, device=device).unsqueeze(


for token in generate_text_basic_stream(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=500,
    eos_token_id=tokenizer.eos_token_id
):
    token_id = token.squeeze(0).tolist()
    print(
        tokenizer.decode(token_id),
        end="",
        flush=True
    )
```

```
<think>
Okay, the user wants a short introduction to MTP, which is Multi Token Predictio
n. First, I need to make sure I understand what MTP is. From what I know, MTP is
a method used in machine learning, particularly in natural language processing, t
o predict the next token in a sequence. It's different from traditional methods l
ike RNNs or Transformers because it uses a single token to predict the next one,
which can be more efficient.

I should start by defining MTP. Then, explain its purpose. Maybe mention that i
t's a technique that can handle long sequences more effectively. Also, highlight
its advantages over other methods. Maybe include some examples, like how it's use
d in tasks like text generation or sequence prediction. Make sure the introductio
n is concise but covers the key points. Avoid technical jargon if possible, but s
till be clear. Check if there's any specific context the user is interested in, b
ut since they didn't specify, keep it general. Also, ensure the tone is informati
ve and helpful.
</think>

MTP (Multi-Token Prediction) is a technique used in machine learning, particularl
y in natural language processing, to predict the next token in a sequence. It lev
erages a single token to generate the next one, making it more efficient than tra
ditional methods that require multiple tokens. This approach is especially useful
for tasks like text generation or sequence prediction, where handling long sequen
ces can be computationally intensive. MTP aims to improve performance by focusing
on the most relevant token in the current context.
```

In [ ]: